Log Analysis with CLI Tools

Kali Linux VirtualBox Evidence Report

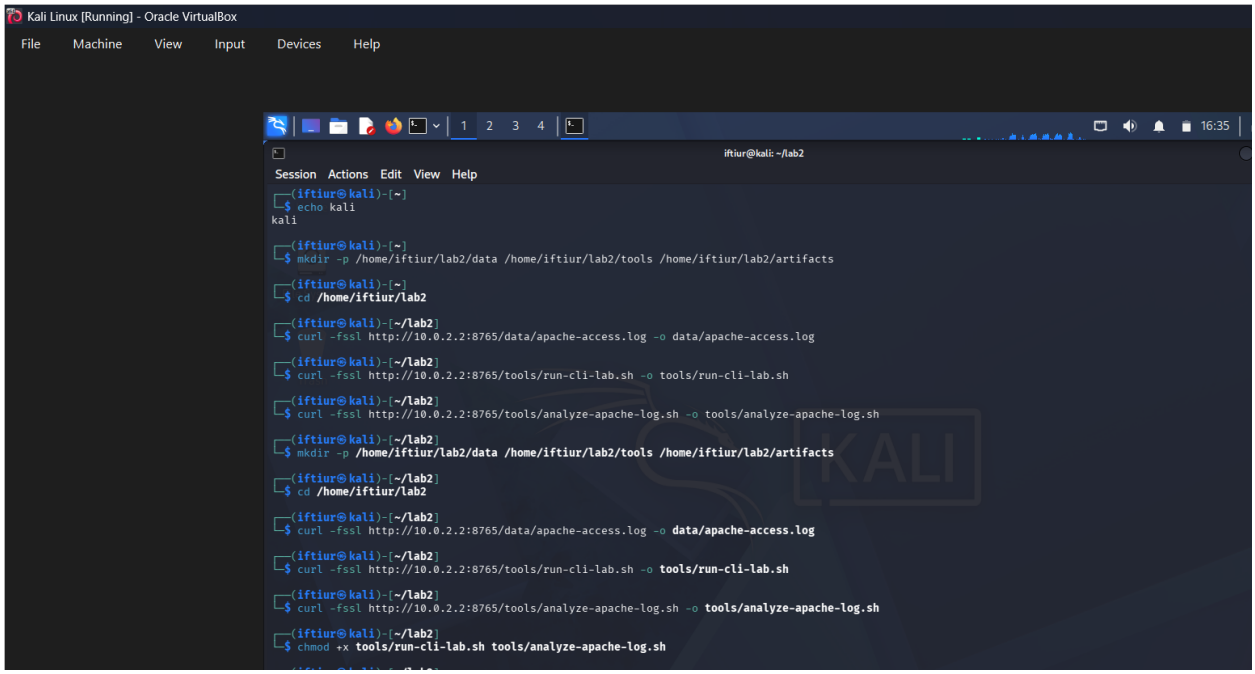
Dataset: repository-provided synthetic Apache access log | Environment: Kali Linux VM | Scope: offline defensive analysis

Execution statement

The commands in this report were executed inside the user-provided Kali Linux VirtualBox VM against the bundled synthetic Apache log. The dataset uses documentation-only addresses and contains no production records, credentials, cookies, or external targets. Suspicious strings are investigative leads, not proof of exploitation, compromise, or attribution.

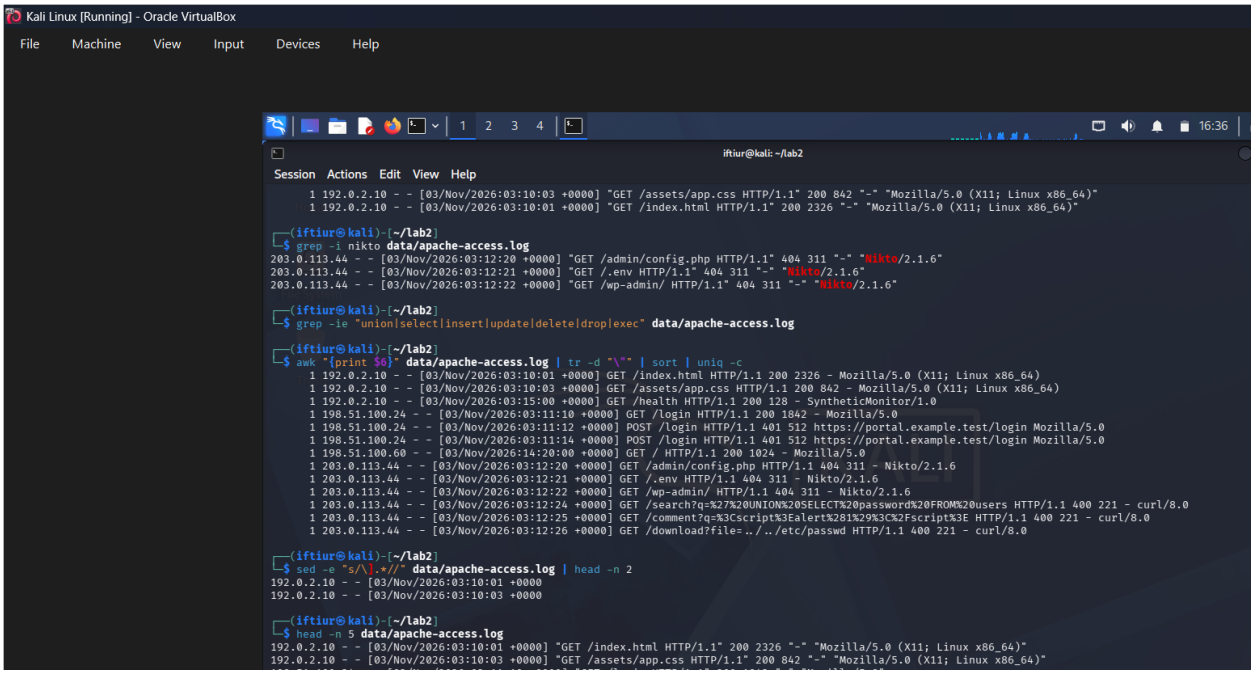
Evidence and explanations

1. Environment setup



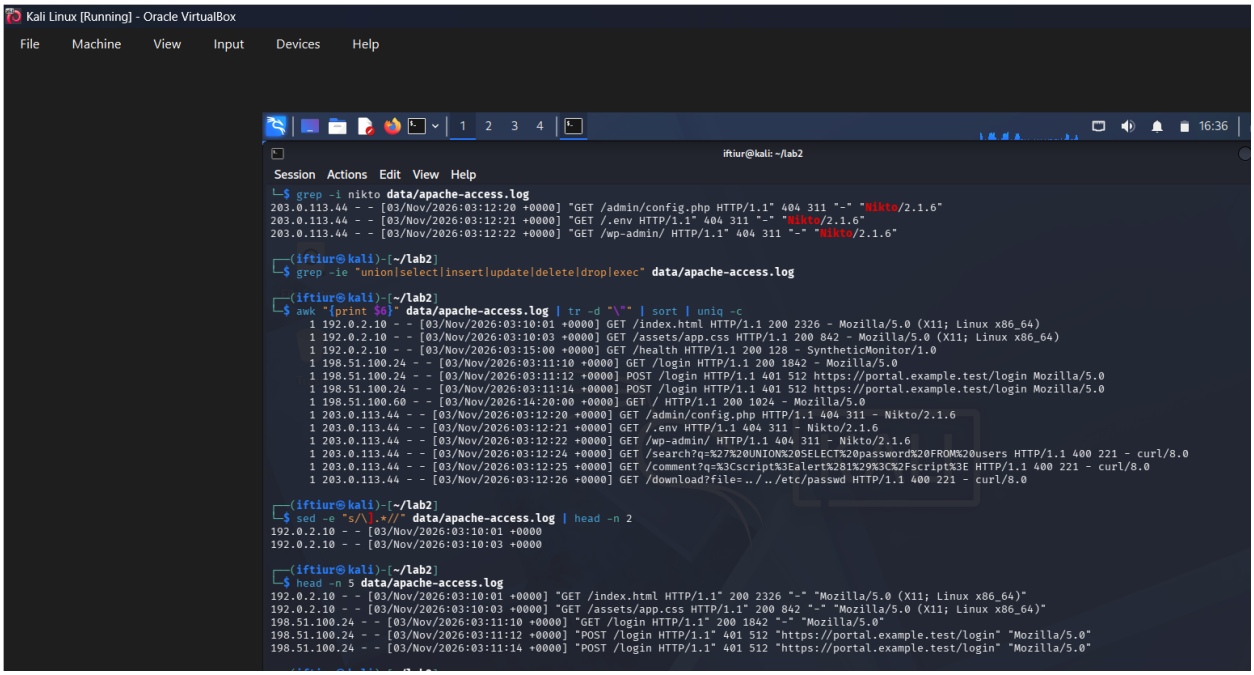
Explanation: The workspace was created in the Kali VM and the repository log and analysis scripts were transferred into it. This confirms the evidence was collected inside Kali rather than simulated on Windows.

2. Inspect the log format



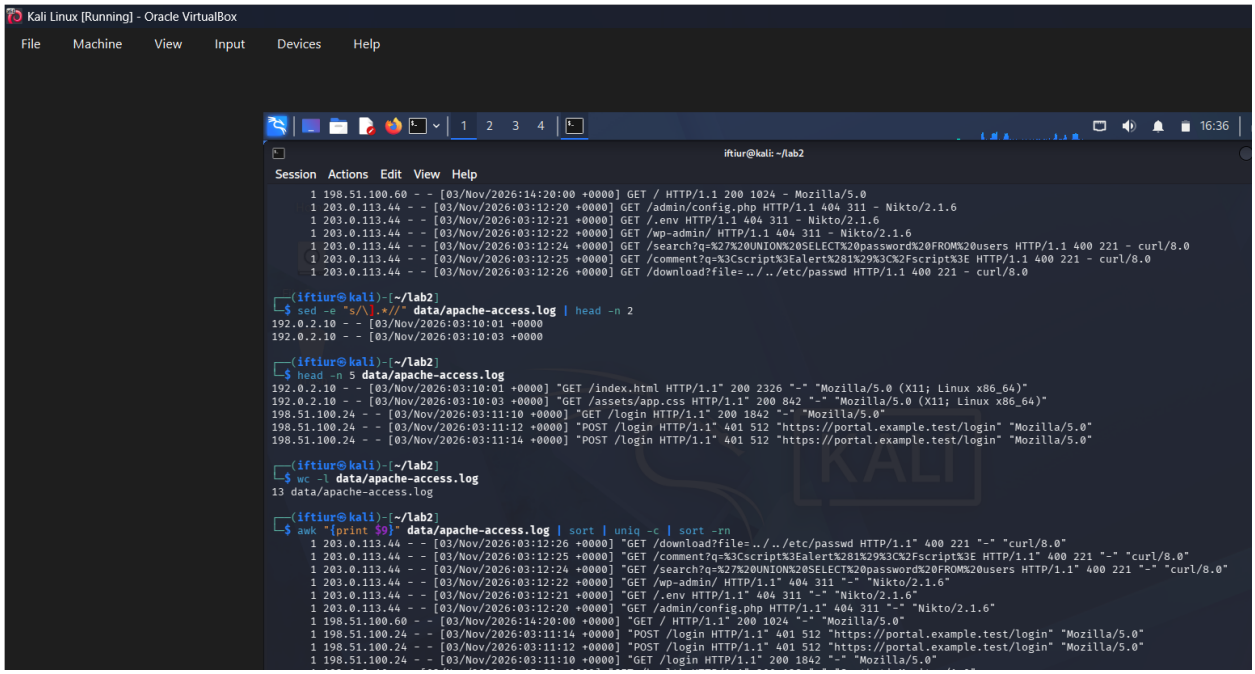
Explanation: The head command displays the first five Apache Combined Log Format records. The fields provide source address, timestamp, request, status, response size, referrer, and user agent for later analysis.

3. Count requests



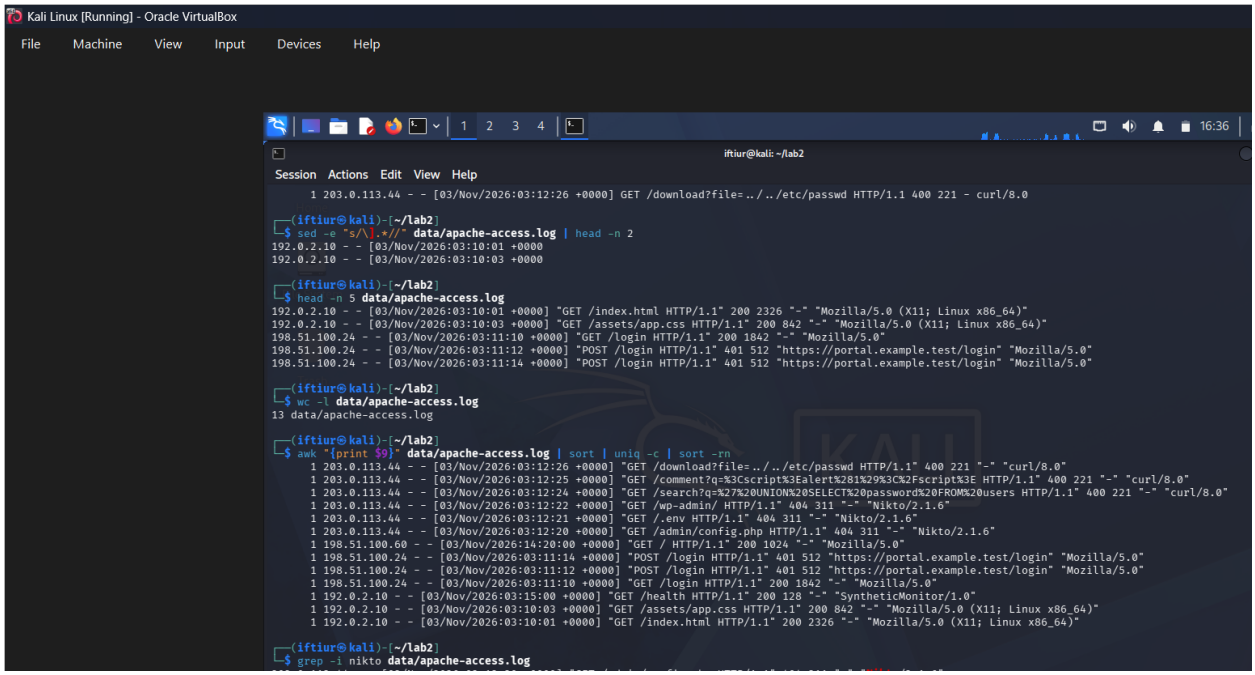
Explanation: wc -l counts the records in apache-access.log. The output shows 13 requests, establishing the small dataset size used for the exercise.

4. Review status codes



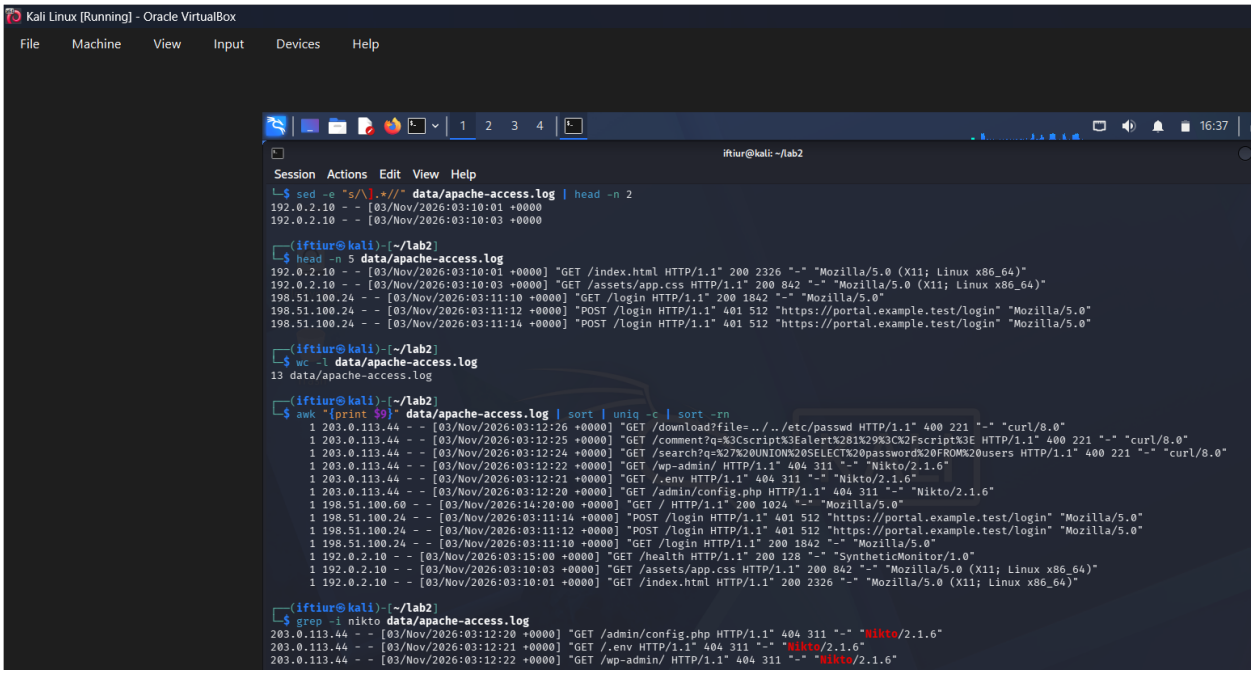
Explanation: awk extracts the status field, while sort and uniq count each value. The distribution identifies successful responses and error responses that require context during triage.

5. Search for scanner activity



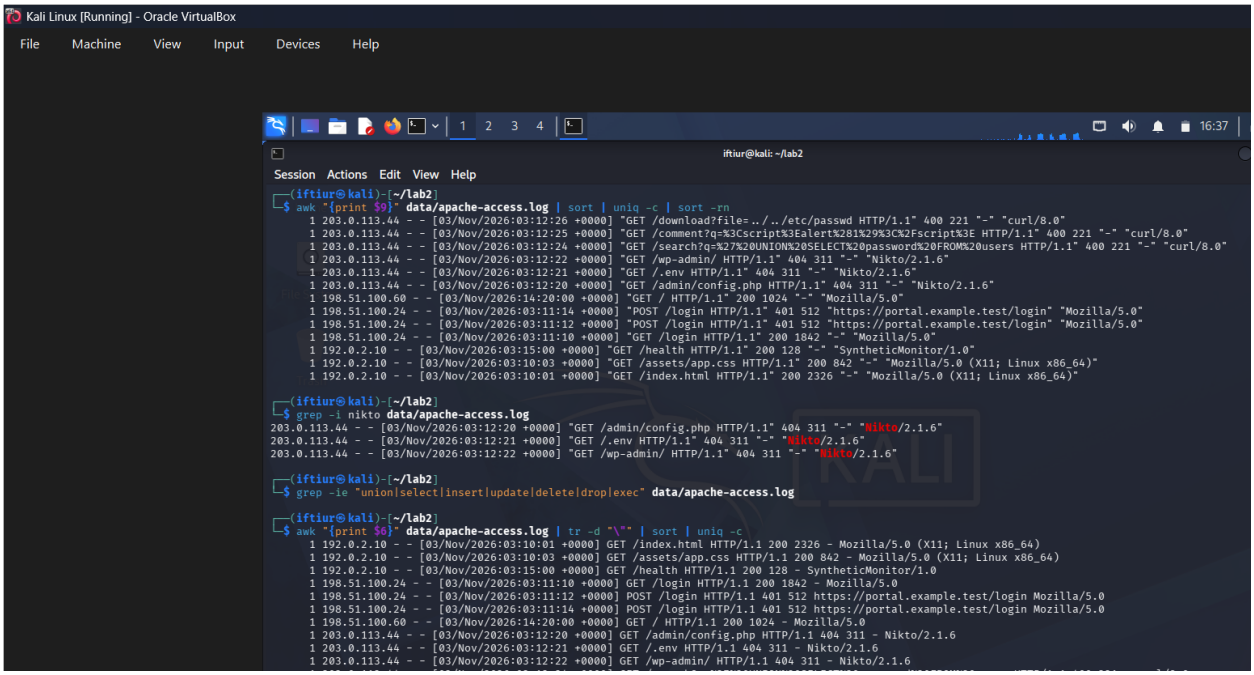
Explanation: grep -i nikto searches case-insensitively for the Nikto user agent. Three matches are scanner-like leads, not proof of unauthorized activity or compromise.

6. Search for SQL-like indicators



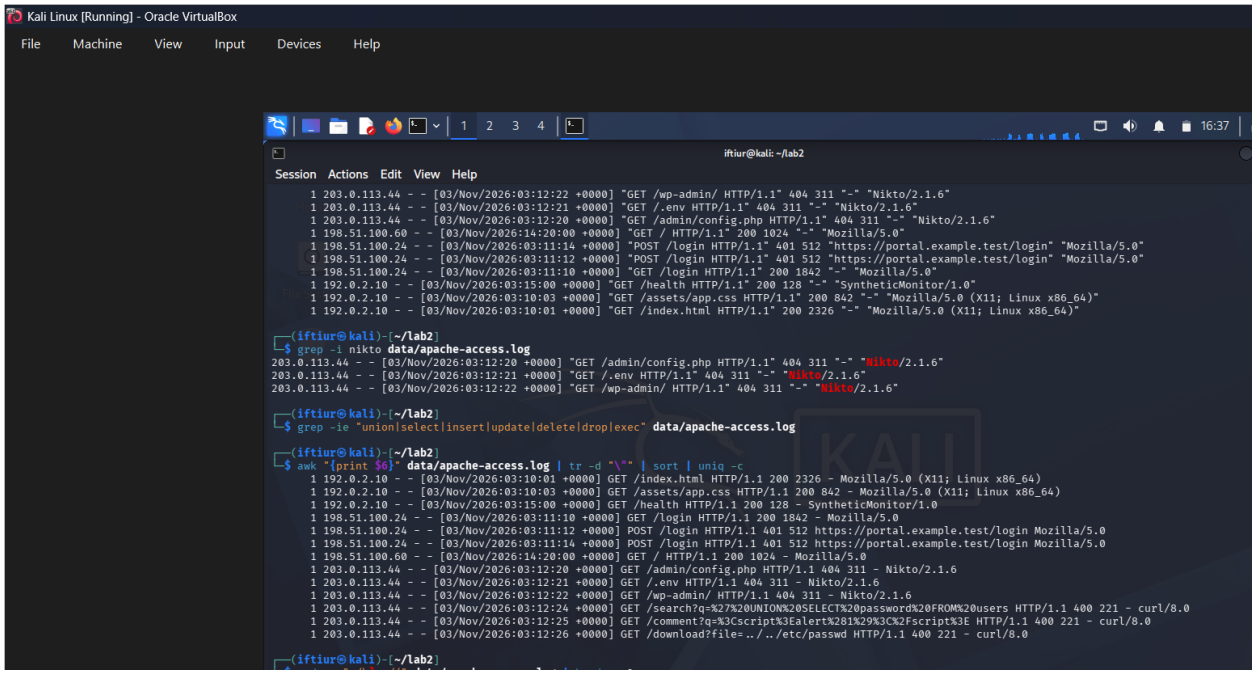
Explanation: grep -iE checks bounded SQL-related terms such as union, select, and drop. The matching request is an investigation lead that should be correlated with application and WAF logs.

7. Count HTTP methods



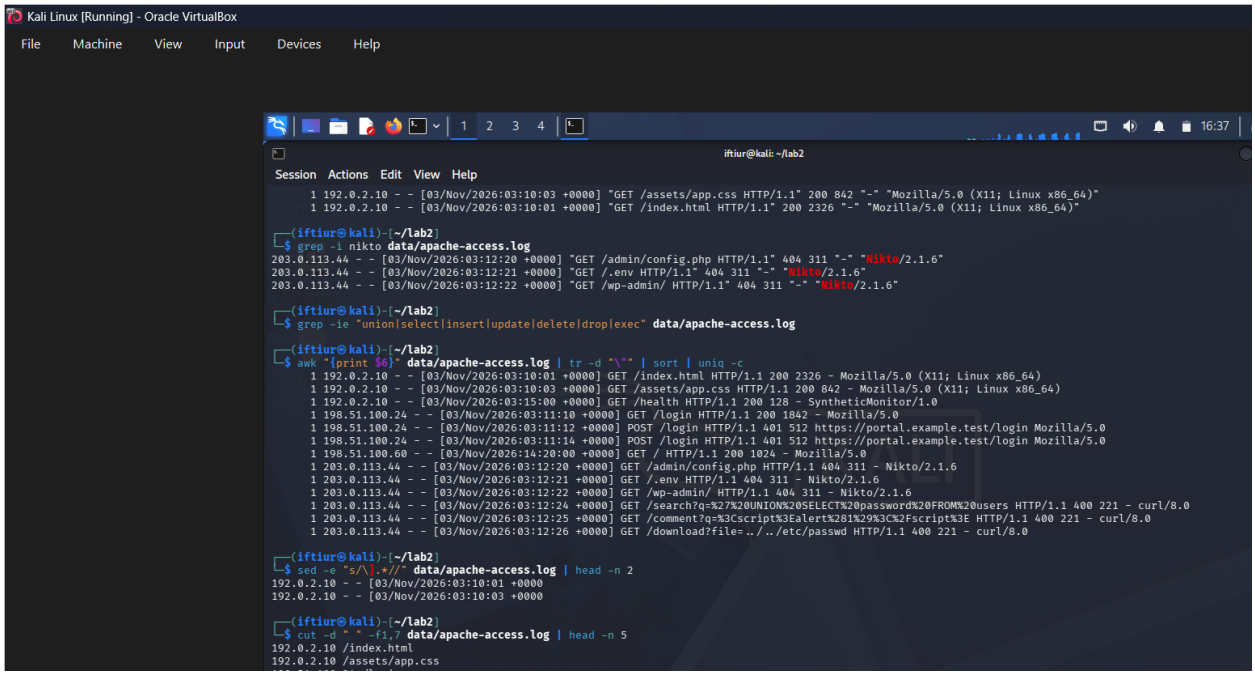
Explanation: awk extracts the request method, tr removes quotation marks, and sort/uniq count methods. The output shows the balance of GET and POST activity in the sample.

8. Trim records with sed



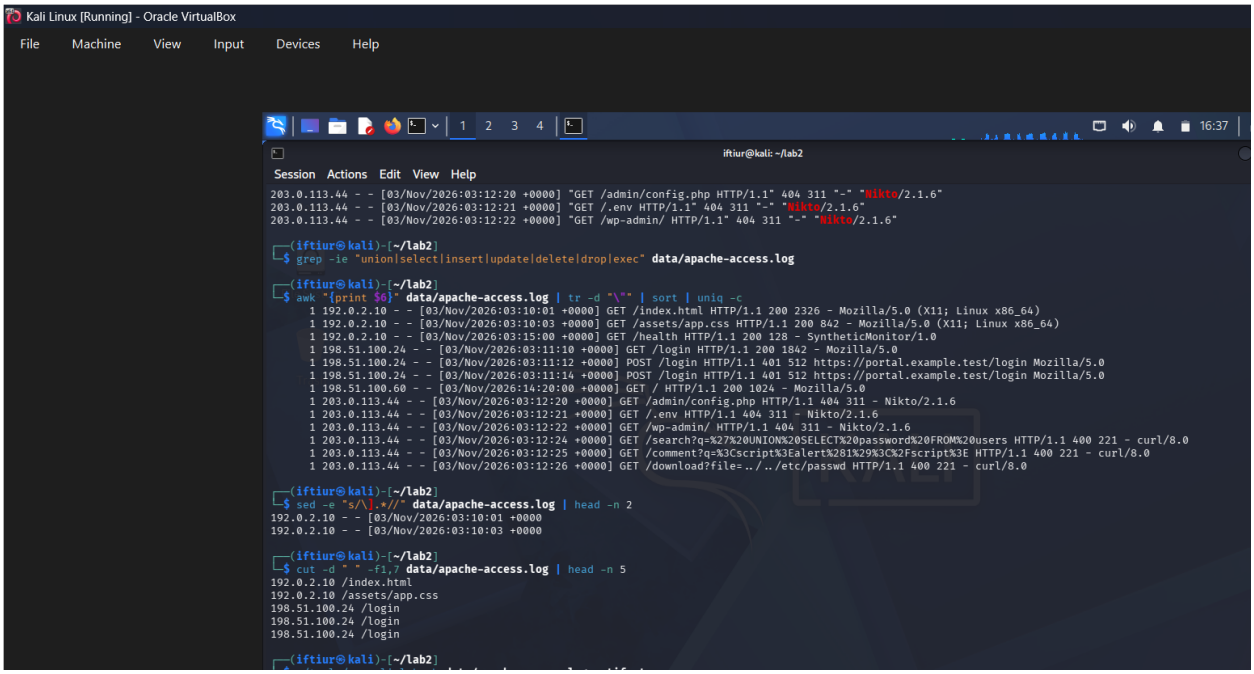
Explanation: sed removes the portion after the timestamp and head limits the display. This creates a compact view for reviewing timing without printing the full record.

9. Extract selected fields



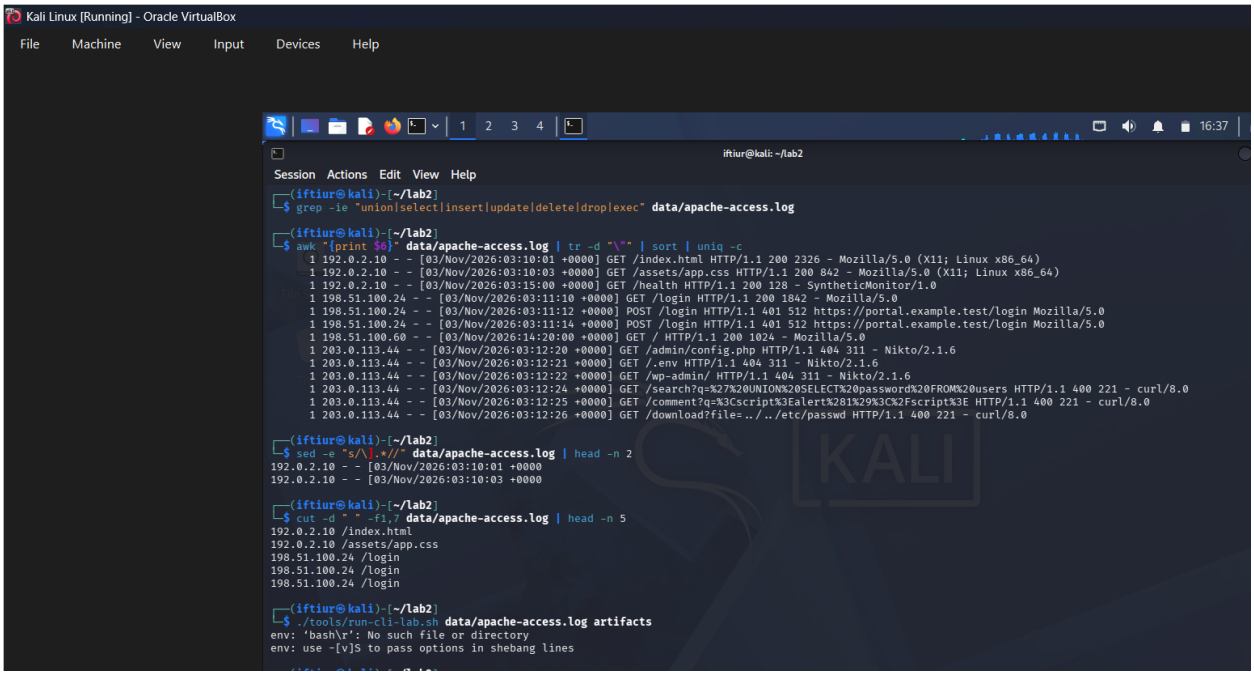
Explanation: cut selects the source address and request field from each record. The output provides a quick relationship between clients and requested paths.

10. Run the complete exercise



Explanation: The run-cli-lab.sh runner executes the complete offline workflow and writes a reproducible report under artifacts. Successful completion confirms the supplied log and scripts work together in Kali.

11. Review generated report



Explanation: cat displays the generated report so the analyst can verify the recorded commands and findings. The report preserves an auditable text record alongside the screenshots.

Conclusion

The Kali execution produced reproducible command evidence for log format inspection, volume counting, status-code review, scanner and SQL-like pattern searches, method analysis, field extraction, and the complete automated runner. Follow-up triage should correlate these leads with application, authentication, and WAF telemetry before drawing conclusions.